APLICACIONES USANDO ASP. NET MVC CORE

1. OBJETIVO. Construir nuestras primeras aplicaciones usando ASP. NET MVC Core

2. RECURSOS A UTILIZAR.

a) Visual Studio 2022: Es una de las mejores IDE para construir aplicaciones multiplataforma para Windows, Mac, Linux,iOS y Android.

b) El Visual Studio 2022 se puede utilizar de manera gratuita por varios meses registrándose con una **cuenta o correo de microsoft.**

3. REVISIÓN DE CONCEPTOS.

a) ASP. NET MVC Core: Framework para construir aplicaciones Webs y APIs usando el patrón de diseño MVC (Modelo, Vista y Controlador).

b) Es un marco de trabajo (Framework) de código abierto y multiplataforma desarrollado por Microsoft para crear aplicaciones web modernas, servicios en la nube y APIs ASP.NET Core.

c) Visualizando las versiones del SDK de .NET

```
C:\Users\upn_i>dotnet --info
SDK DE .NET:
 Version:   7.0.201
 Commit:    68f2d7e7a3

Entorno de tiempo de ejecución:
 OS Name:     Windows
 OS Version:  10.0.19045
 OS Platform: Windows
 RID:         win10-x64
 Base Path:   C:\Program Files\dotnet\sdk\7.0.201\

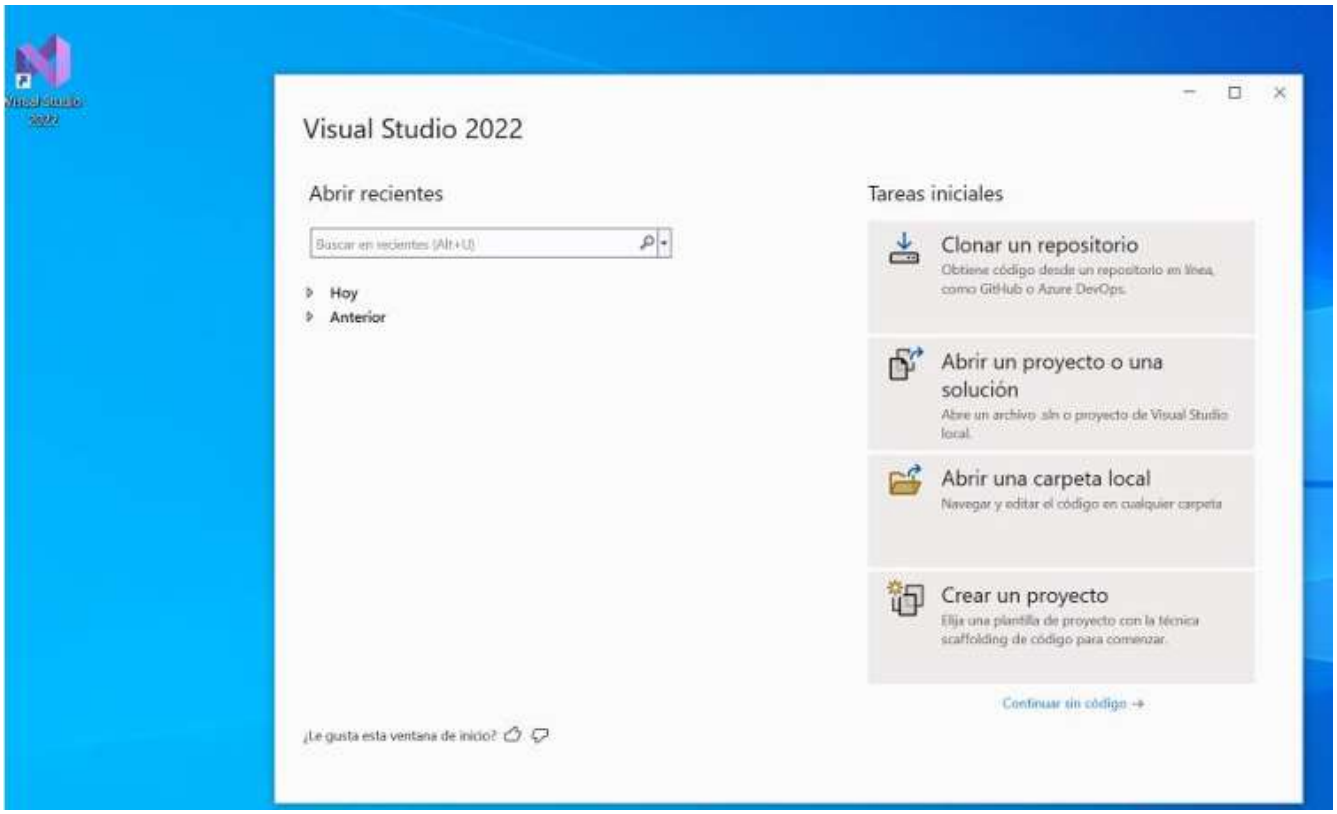
Host:
  Version:      7.0.3
  Architecture: x64
  Commit:       0a2bda10e8

.NET SDKs installed:
  7.0.201 [C:\Program Files\dotnet\sdk]

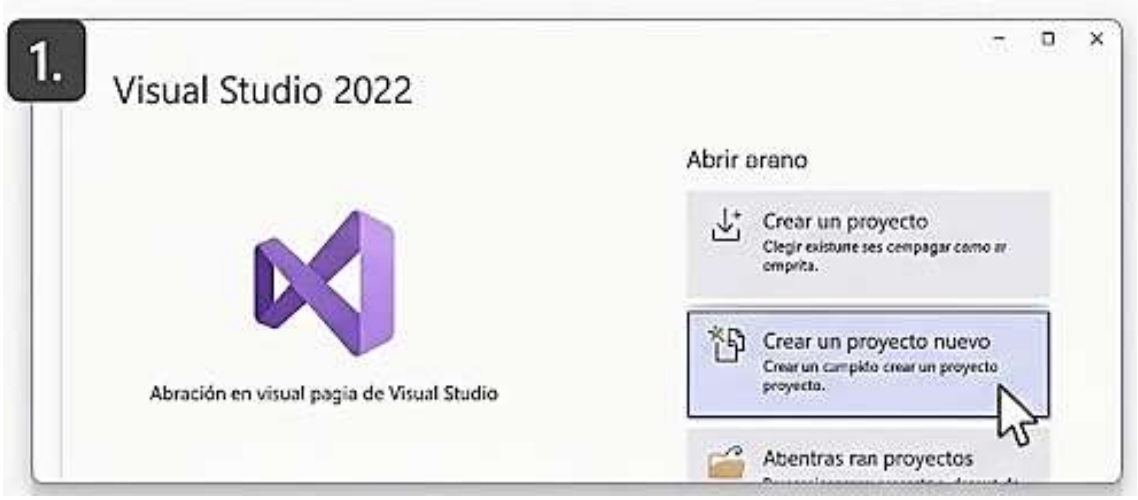
.NET runtimes installed:
  Microsoft.AspNetCore.App 6.0.14 [C:\Program Files\dotnet\shared\Microsoft.AspNetCore.App]
  Microsoft.AspNetCore.App 7.0.3 [C:\Program Files\dotnet\shared\Microsoft.AspNetCore.App]
  Microsoft.NETCore.App 6.0.14 [C:\Program Files\dotnet\shared\Microsoft.NETCore.App]
  Microsoft.NETCore.App 7.0.3 [C:\Program Files\dotnet\shared\Microsoft.NETCore.App]
  Microsoft.WindowsDesktop.App 6.0.14 [C:\Program Files\dotnet\shared\Microsoft.WindowsDesktop.App]
  Microsoft.WindowsDesktop.App 7.0.3 [C:\Program Files\dotnet\shared\Microsoft.WindowsDesktop.App]
```

3. DESARROLLO DE APLICACIONES:

1. Iniciar el Visual Studio 2022, clic en Crear un proyecto



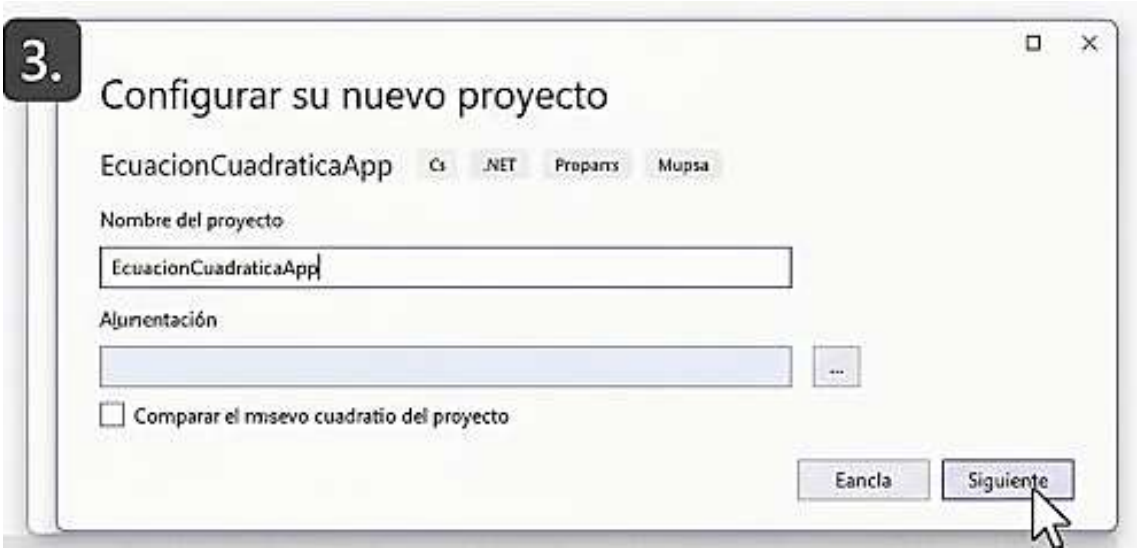
2. Al seleccionar **Crear un proyecto nuevo**.



3. Buscar y seleccionar la plantilla **Aplicación Web de ASP.NET Core (Razor Pages)**.



4. Asignar el nombre EcuacionCuadraticaApp al proyecto y presionar **Siguiente**.



5. Seleccionar el marco **.NET 8.0** o **.NET 9.0** y hacer clic en **Crear**.



3. MODELO DE LA PÁGINA (Pages/Index.cshtml.cs**)**

Reemplazar el contenido de **Index.cshtml.cs** con el código C# que procesa los coeficientes y evalúa los casos de la discriminante: $(\Delta = b^2 - 4ac)$

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using System.ComponentModel.DataAnnotations;

namespace EcuacionCuadraticaApp.Pages
{
    public class IndexModel : PageModel
    {
        [BindProperty]
        [Required(ErrorMessage = "El coeficiente 'a' es requerido.")]
        public double CoeficienteA { get; set; }

        [BindProperty]
        [Required(ErrorMessage = "El coeficiente 'b' es requerido.")]
        public double CoeficienteB { get; set; }

        [BindProperty]
        [Required(ErrorMessage = "El coeficiente 'c' es requerido.")]
```

```
        public double CoeficienteC { get; set; }

        public bool Calculado { get; set; } = false;
        public double Discriminante { get; set; }
        public string ResultadoX1 { get; set; } = string.Empty;
        public string ResultadoX2 { get; set; } = string.Empty;
        public string TipoSolucion { get; set; } = string.Empty;
        public string? MensajeError { get; set; }

        public void OnGet()
        {
        }

        public void OnPost()
        {
            if (!ModelState.IsValid)
            {
                return;
            }

            // Validación de ecuación cuadrática (a != 0)
            if (CoeficienteA == 0)
            {
               MensajeError = "El coeficiente 'a' no puede ser igual a cero. "
                            + Si 'a = 0', no es una ecuación cuadrática.";
                return;
            }

            double a = CoeficienteA;
            double b = CoeficienteB;
            double c = CoeficienteC;

            // Cálculo del Discriminante (b^2 - 4ac)
            Discriminante = Math.Pow(b, 2) - (4 * a * c);
            Calculado = true;

           // Caso 1: Discriminante > 0 (Dos soluciones reales y distintas)
```

```
            if (Discriminante > 0)
            {
                double x1 = (-b + Math.Sqrt(Discriminante)) / (2 * a);
                double x2 = (-b - Math.Sqrt(Discriminante)) / (2 * a);

                ResultadoX1 = x1.ToString("F4");
                ResultadoX2 = x2.ToString("F4");
                TipoSolucion = "Dos raíces reales distintas";
            }
            // Caso 2: Discriminante == 0 (Una solución real doble)
            else if (Discriminante == 0)
            {
                double x = -b / (2 * a);

                ResultadoX1 = x.ToString("F4");
                ResultadoX2 = x.ToString("F4");
                TipoSolucion = "Una raíz real doble";
            }
           // Caso 3: Discriminante < 0 (Dos soluciones complejas conjugadas)
            else
            {
                double parteReal = -b / (2 * a);
               double parteImaginaria = Math.Sqrt(-Discriminante) / (2 * a);

                ResultadoX1 = $"{parteReal:F4} + {parteImaginaria:F4}i";
                ResultadoX2 = $"{parteReal:F4} - {parteImaginaria:F4}i";
                TipoSolucion = "Dos raíces complejas conjugadas";
            }
        }
    }
}
```

4. VISTA DE LA PÁGINA (Pages/Index.cshtml)

Reemplazar el contenido de **Index.cshtml** con la interfaz en Razor y HTML5 estilizada con Bootstrap:

```
Razor CSHTML
```

```
@page
@model IndexModel
@{
    ViewData["Title"] = "Resolución de Ecuación Cuadrática";
}

<div class="container mt-5" style="max-width: 600px;">
    <div class="card shadow-sm">
        <div class="card-header bg-primary text-white text-center">
            <h3 class="mb-0">Ecuación Cuadrática</h3>
            <p class="mb-0 fs-6">Forma: <code>ax² + bx + c = 0</code></p>
        </div>
        <div class="card-body p-4">
            <form method="post">
                <div class="row g-3">
                    <div class="col-md-4">
                        <label asp-for="CoeficienteA" class="form-label fw-bold">
                            Coeficiente (a)</label>
                        <input asp-for="CoeficienteA" type="number" step="any"
                            class="form-control" placeholder="Ej: 1" />
                    <span asp-validation-for="CoeficienteA" class="text-danger small"></span>
                    </div>
                    <div class="col-md-4">
                        <label asp-for="CoeficienteB" class="form-label fw-bold">
                               Coeficiente (b)</label>
                        <input asp-for="CoeficienteB" type="number" step="any"
                               class="form-control" placeholder="Ej: -3" />
                    <span asp-validation-for="CoeficienteB" class="text-danger small"></span>
                    </div>
                    <div class="col-md-4">
                        <label asp-for="CoeficienteC" class="form-label fw-bold">
                               Coeficiente (c)</label>
                        <input asp-for="CoeficienteC" type="number" step="any"
                               class="form-control" placeholder="Ej: 2" />
                    <span asp-validation-for="CoeficienteC" class="text-danger small"></span>
                    </div>
                </div>

                <div class="d-grid gap-2 mt-4">
                 <button type="submit" class="btn btn-primary btn-lg">Calcular Raíces</button>
                </div>
```

```
            </form>

            @if (!string.IsNullOrEmpty(Model.MensajeError))
            {
                <div class="alert alert-danger mt-4 text-center" role="alert">
                    @Model.MensajeError
                </div>
            }

            @if (Model.Calculado && string.IsNullOrEmpty(Model.MensajeError))
            {
                <hr class="my-4" />
                <h4 class="text-center text-secondary mb-3">Resultados</h4>
                <ul class="list-group">
                 <li class="list-group-item d-flex justify-content-between align-items-center">
                        <span><strong>Discriminante (Δ):</strong></span>
                        <span class="badge bg-secondary fs-6">
                            @Model.Discriminante.ToString("F4")</span>
                    </li>
                    <li class="list-group-item d-flex justify-content-between
                            align-items-center">
                        <span><strong>Tipo de Solución:</strong></span>
                        <span class="text-muted">@Model.TipoSolucion</span>
                    </li>
                 <li class="list-group-item d-flex justify-content-between align-items-center">
                        <span><strong>x₁:</strong></span>
                        <span class="text-primary fw-bold fs-5">
                             @Model.ResultadoX1</span>
                    </li>
                    <li class="list-group-item d-flex justify-content-between
                             align-items-center">
                        <span><strong>x₂:</strong></span>
                      <span class="text-primary fw-bold fs-5">@Model.ResultadoX2</span>
                    </li>
                </ul>
            }
        </div>
    </div>
</div>
```

```
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

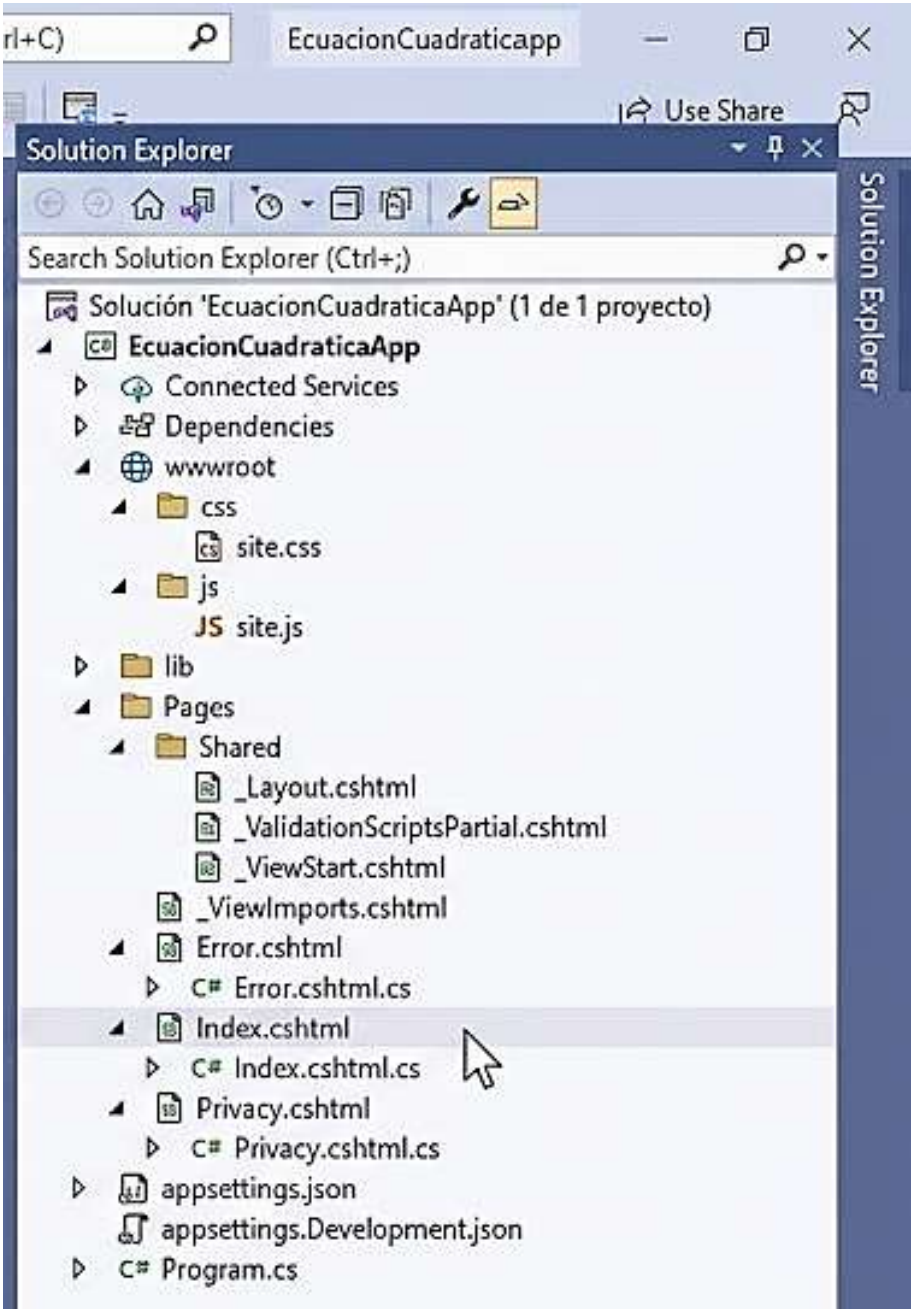
5. LÓGICA DE NEGOCIO CUBIERTA

Caso Matemático	Condición	Comportamiento del Código
No Cuadrática	$a = 0$	Muestra un mensaje de advertencia impidiendo la división por cero.
Raíces Reales Distintas	$\Delta > 0$	Aplica $x = \frac{-b \pm \sqrt{\Delta}}{2a}$ obteniendo dos números reales.
Raíz Real Doble	$\Delta = 0$	Calcula $x = \frac{-b}{2a}$, x_1 y x_2 obtienen el mismo valor.
Raíces Complejas	$\Delta < 0$	Separa la parte $(\frac{-b}{2a})$ real y la parte imaginaria $(\frac{\sqrt{-\Delta}}{2a})$ formateando el resultado como $a \pm bi.$

Resultado:



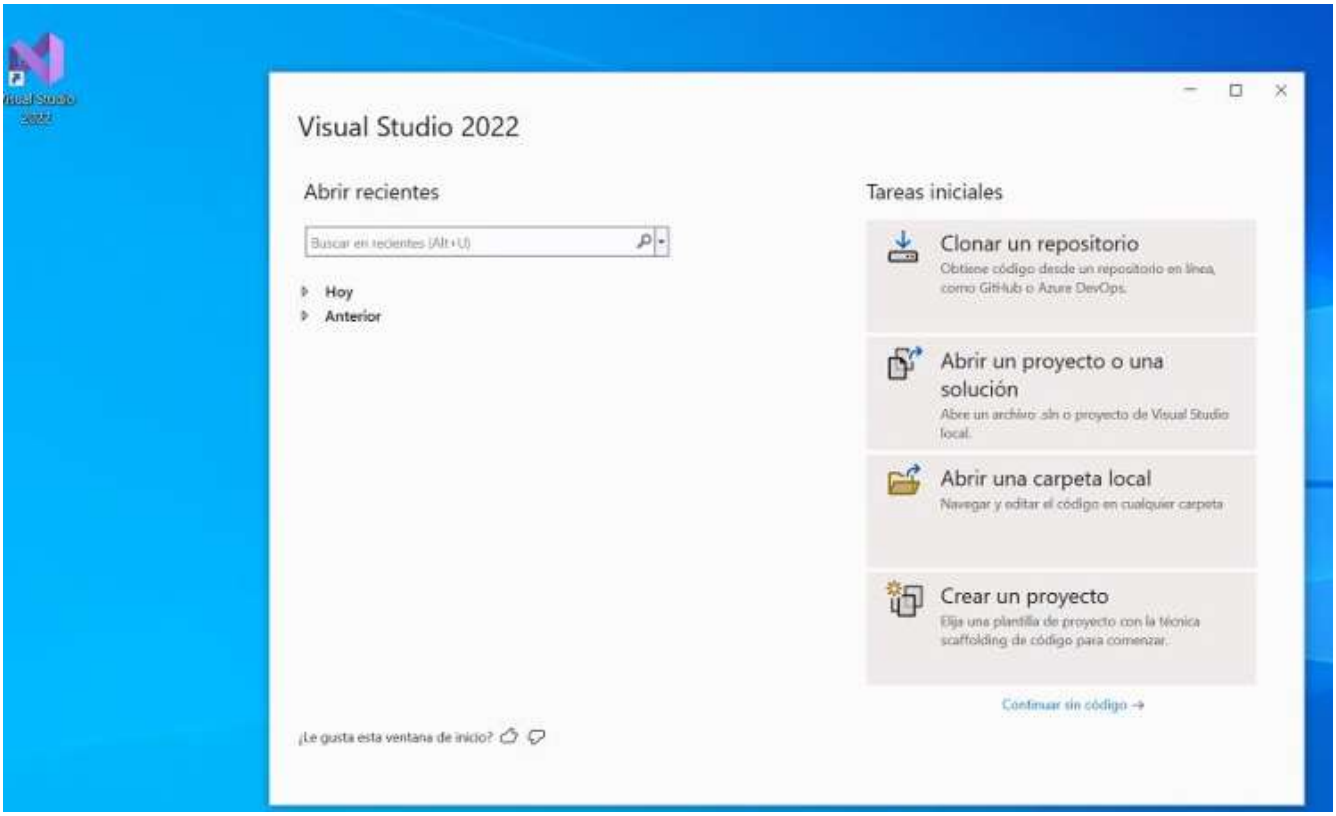
Estructura del Proyecto:



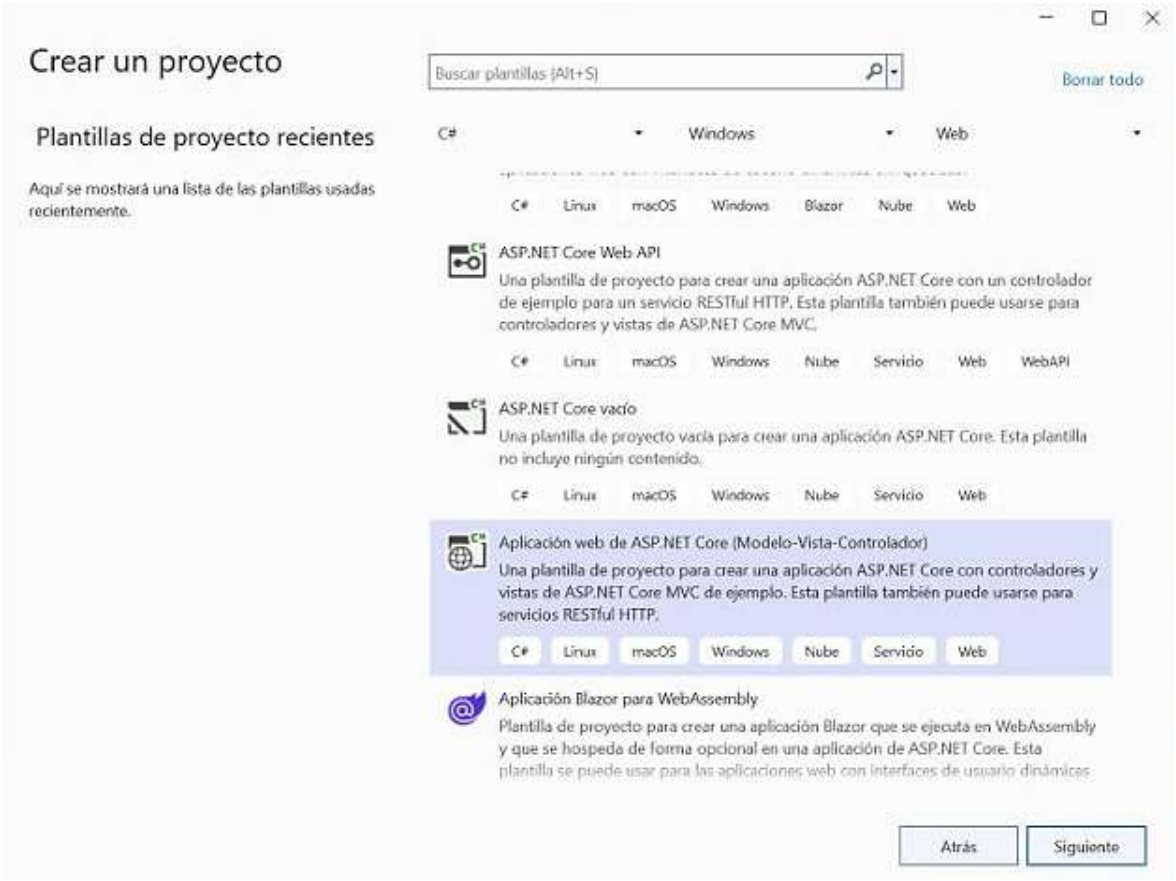
MODELO MVC

El patrón Modelo-Vista-Controlador (MVC) es el patrón arquitectónico estándar para crear aplicaciones web modernas en ASP.NET Core. Es un patrón de diseño que divide una aplicación en tres componentes lógicos principales: el **Modelo**, la **Vista** y el **Controlador**.
Esta separación ayuda a gestionar la complejidad al crear aplicaciones, lo que permite centrarse en un aspecto de la implementación a la vez.

1. Iniciar el Visual Studio 2022, clic en Crear un proyecto



2. Seleccionar la plantilla: Aplicación web de ASP. NET Core (Model-Vista-Controlador), clic en Siguiente:



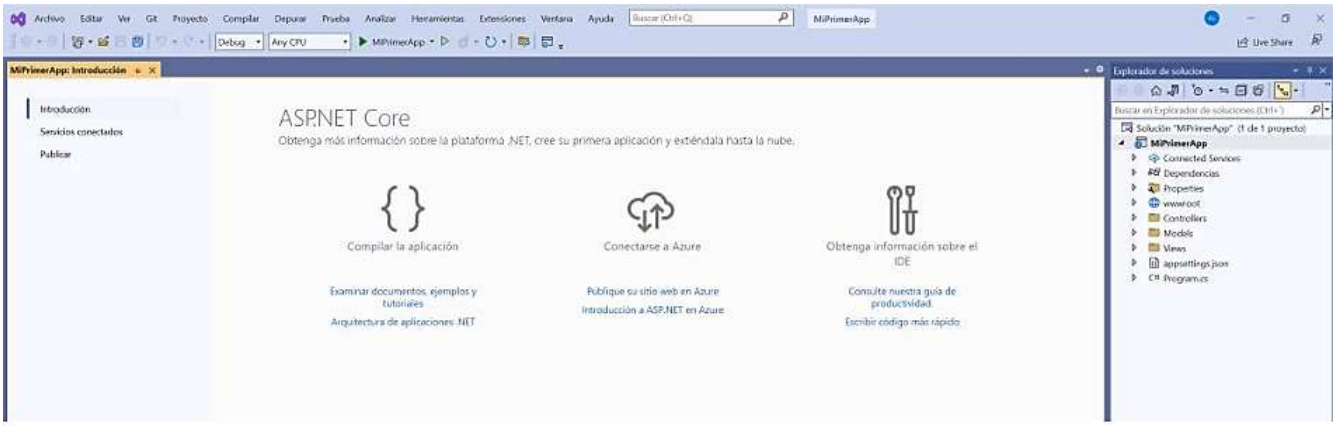
3. Configurar el proyecto, para ello se ingresa el nombre y la ubicación del proyecto, y clic en Siguiente.



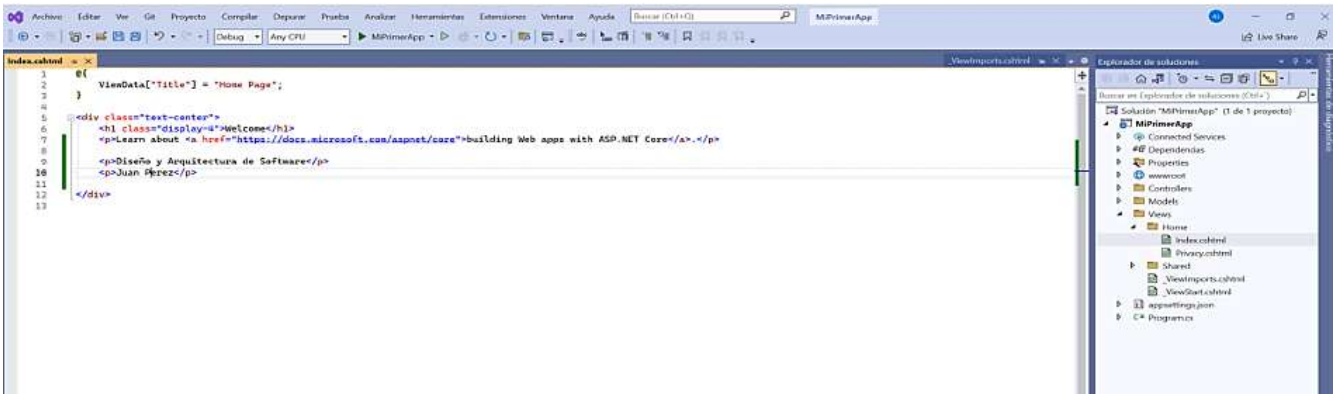
4. En la siguiente pantalla dejar las opciones por defecto y clic en Crear

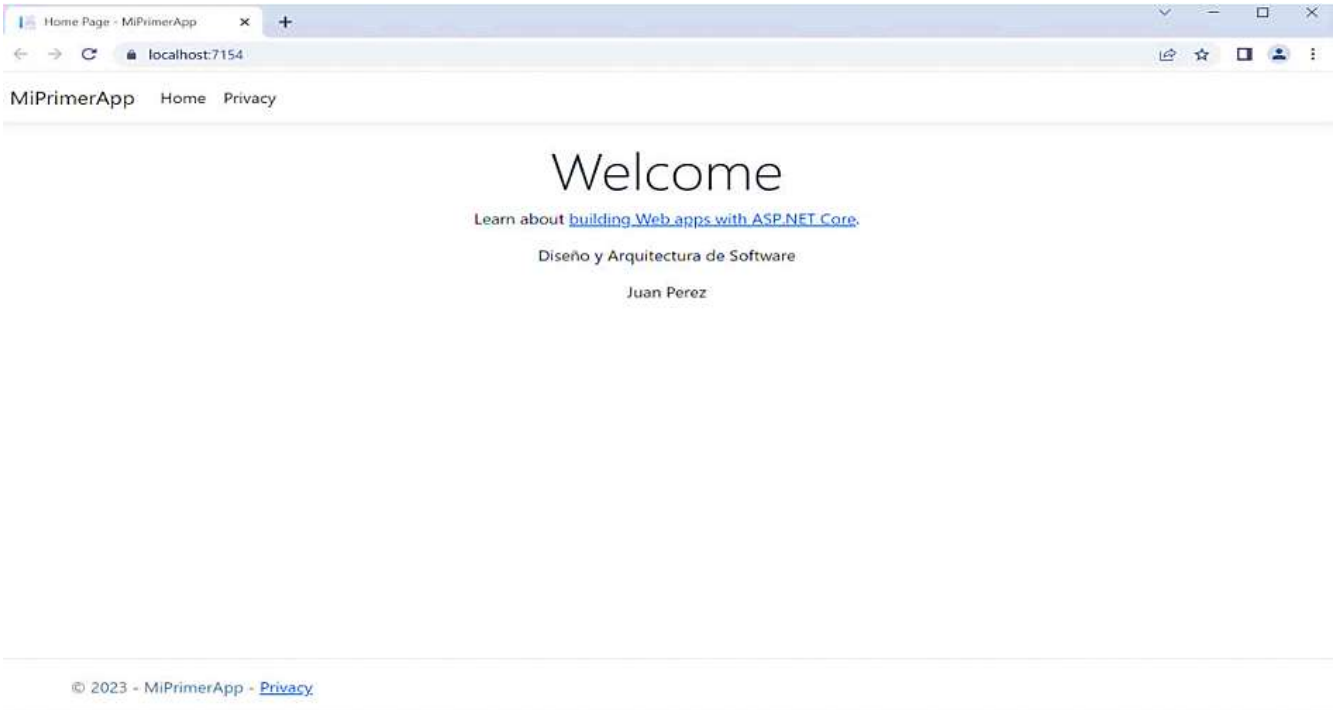


5. Se construirá el proyecto con la estructura de directorios MVC.

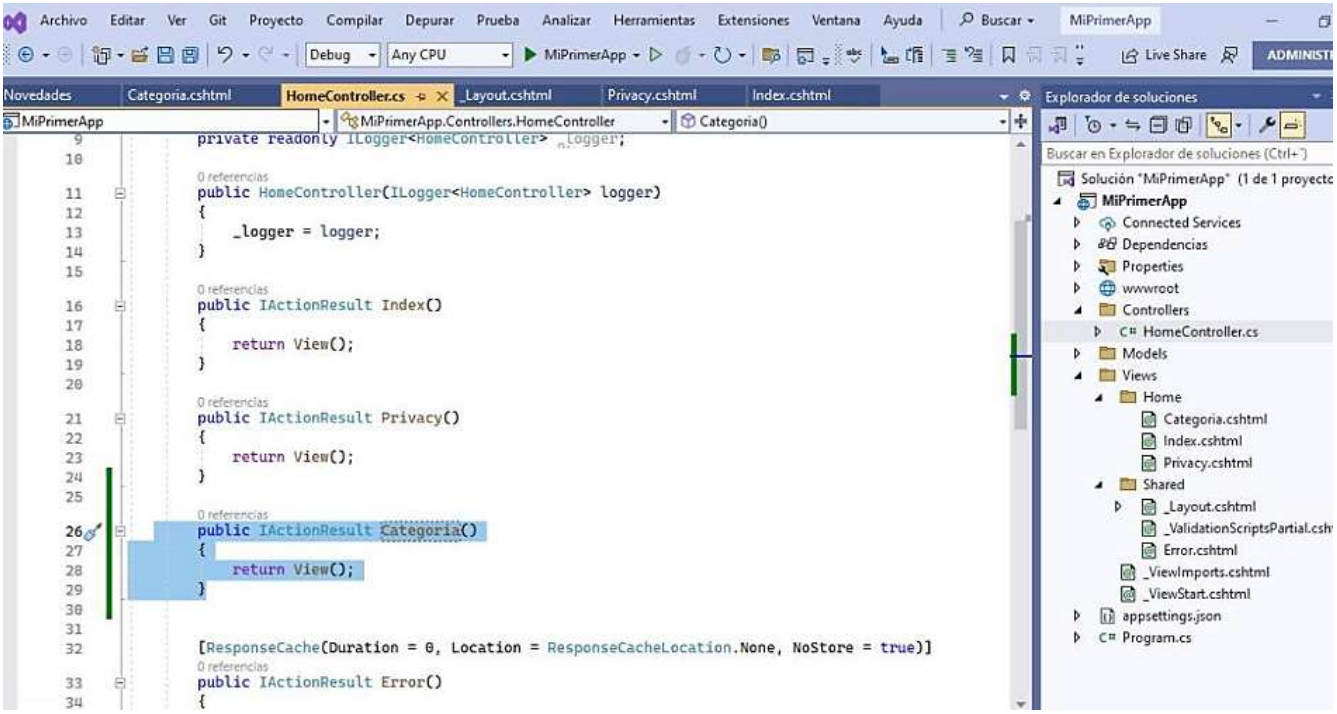


6. Modificar el archivo /Views/Home/Index.cshtml , agregar el nombre del curso, su nombre y ejecutar el proyecto: ▶ MiPrimerApp

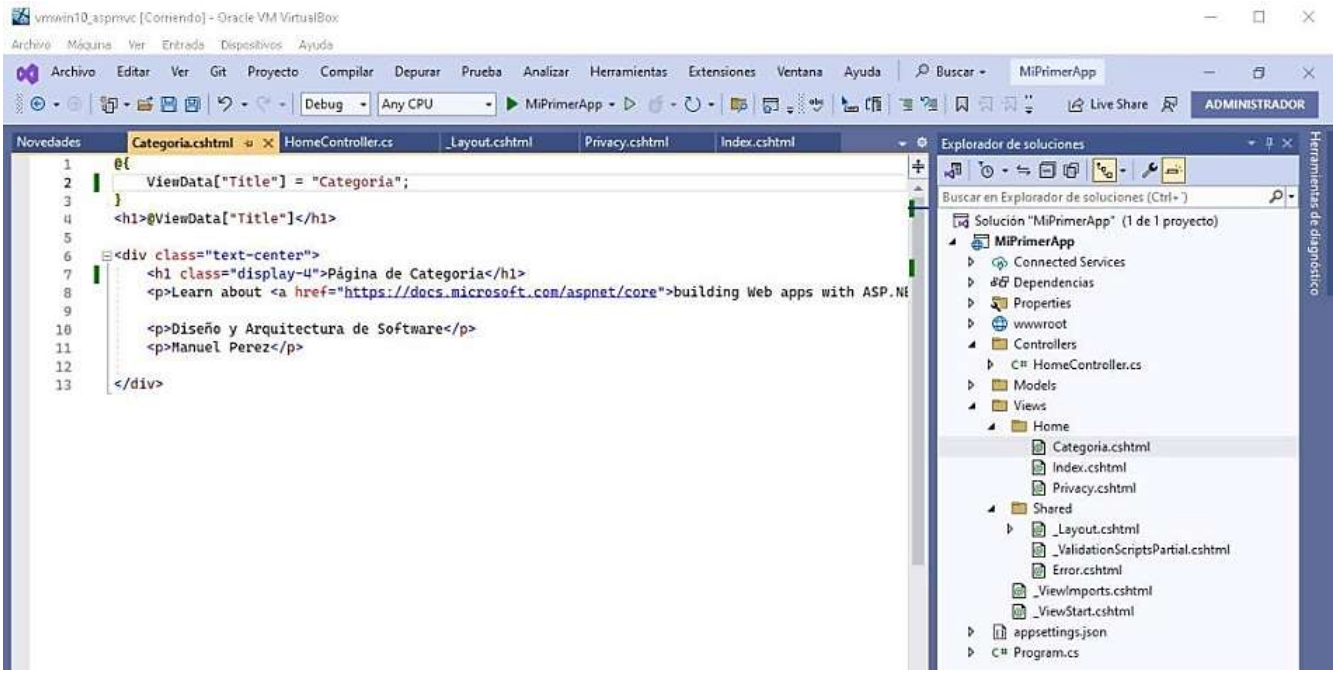




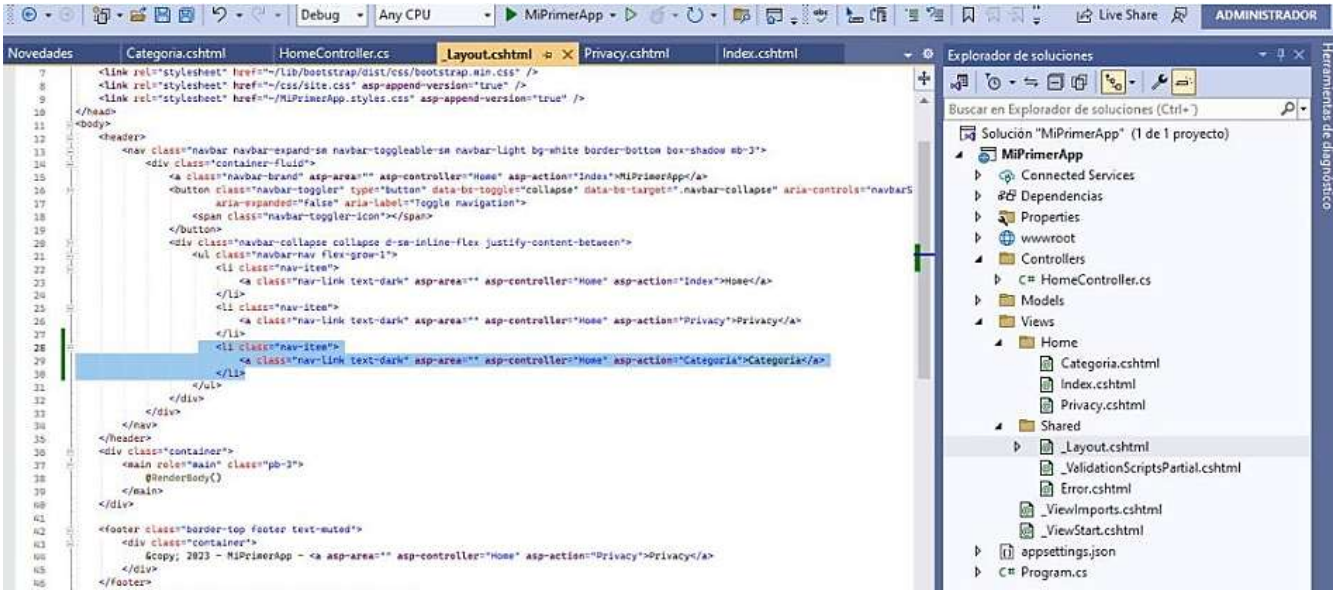
7. Agregar nuevas páginas web. Para ello agregar el siguiente código en el controlador Home.



8. Clic derecho a Categoria dentro del controlador y agregar la vista. Agregar el codigo como se muestra en la imagen:



9. Modificar el Shared/_Layout.cshtml con esta nueva página o menú:



10. Ejecutar y verificar la nueva página web o nuevo menú Categoría:

